

01_schema-rule-py

#python #api #fastapi #swagger #pydantic #backend #SIEM

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ schemas/  
            └─ rule.py
```

El archivo `rule.py` se encuentra dentro de la carpeta de schemas del backend:

```
backend/app/schemas/
```

Este archivo define los schemas relacionados con las reglas de detección del laboratorio SIEM MVP.

En concreto, contiene dos clases principales:

```
RuleCreate  
RuleOut
```

Estas clases no crean tablas directamente en PostgreSQL. Su función es validar los datos que llegan a la API y definir cómo se devuelven las reglas en las respuestas HTTP.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,320p' backend/app/schemas/rule.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,320p'
```

Indica que se impriman las líneas de la 1 a la 320.

```
backend/app/schemas/rule.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
from datetime import datetime  
from typing import Any, Optional
```

```

from pydantic import BaseModel, Field

class RuleCreate(BaseModel):
    name: str = Field(min_length=1, max_length=120)
    enabled: bool = True
    source: Optional[str] = Field(default=None, max_length=64)
    severity_min: Optional[int] = Field(default=None, ge=0, le=10)
    contains: Optional[str] = Field(default=None, max_length=200)

    # throttle por regla (segundos)
    # None => usar DEFAULT_THROTTLE_SECONDS en el motor
    # 0     => sin throttle (alertar siempre)
    throttle_seconds: Optional[int] = Field(default=None, ge=0, le=86400)

    # threshold: dispara alerta cuando hay N eventos que matchean en X segundos
    threshold_count: Optional[int] = Field(default=None, ge=1, le=100000)
    threshold_seconds: Optional[int] = Field(default=None, ge=1, le=86400)

    # match por meta (exacto). Ej: {"host":"kali","facility":"auth"}
    meta_match: Optional[dict[str, Any]] = None

class RuleOut(BaseModel):
    id: int
    name: str
    enabled: bool
    source: Optional[str]
    severity_min: Optional[int]
    contains: Optional[str]

    throttle_seconds: Optional[int]
    threshold_count: Optional[int]
    threshold_seconds: Optional[int]

    meta_match: Optional[dict[str, Any]]
    created_at: datetime

    model_config = {"from_attributes": True}

```

4 Función general del archivo

El archivo `schemas/rule.py` define la estructura de entrada y salida de las reglas de detección.

La clase:

```
RuleCreate
```

se utiliza para validar los datos que llegan al endpoint:

```
POST /rules
```

La clase:

```
RuleOut
```

se utiliza para definir cómo se devuelven las reglas desde la API.

La relación general es:

```

JSON enviado por el cliente
↓
RuleCreate

```

```
↓
modelo SQLAlchemy Rule
↓
tabla rules en PostgreSQL
↓
RuleOut
↓
respuesta JSON
```

Este archivo es importante porque establece qué puede configurar el usuario cuando crea una regla.

5 Estructura general del archivo

El archivo puede dividirse en cuatro bloques:

```
from datetime import datetime
```

Importación del tipo `datetime`.

```
from typing import Any, Optional
```

Importación de tipos opcionales y flexibles.

```
from pydantic import BaseModel, Field
```

Importación de herramientas de Pydantic.

```
class RuleCreate(BaseModel):
    ...
```

Schema de entrada para crear reglas.

```
class RuleOut(BaseModel):
    ...
```

Schema de salida para devolver reglas.

Visualmente:

```
rule.py
├─ Importaciones
├─ RuleCreate
│   ├── name
│   ├── enabled
│   ├── source
│   ├── severity_min
│   ├── contains
│   ├── throttle_seconds
│   ├── threshold_count
│   ├── threshold_seconds
│   └─ meta_match
└─ RuleOut
    ├── id
    ├── name
    ├── enabled
    ├── source
    ├── severity_min
    ├── contains
    ├── throttle_seconds
    ├── threshold_count
    ├── threshold_seconds
    └─ meta_match
```

```
| created_at
| model_config
```

6 Análisis línea por línea

Importación de `datetime`

```
from datetime import datetime
```

Esta línea importa `datetime` desde el módulo estándar `datetime`.

Se utiliza en el schema `RuleOut`:

```
created_at: datetime
```

El campo `created_at` representa la fecha de creación de la regla.

Este valor no lo envía el usuario al crear una regla. Lo genera la base de datos mediante el modelo `Rule`.

Importación de `Any` y `Optional`

```
from typing import Any, Optional
```

Esta línea importa dos tipos desde el módulo estándar `typing`.

`Any`

```
Any
```

Representa cualquier tipo de dato.

En este archivo se usa en:

```
dict[str, Any]
```

Esto permite que `meta_match` sea un diccionario con claves de texto y valores de cualquier tipo.

Ejemplo:

```
{
  "host": "server-01",
  "facility": "auth",
  "attempts": 5
}
```

`Optional`

```
Optional
```

Indica que un campo puede tener un valor o puede ser `None`.

En este archivo se usa en campos como:

```
source: Optional[str]
severity_min: Optional[int]
contains: Optional[str]
throttle_seconds: Optional[int]
threshold_count: Optional[int]
```

```
threshold_seconds: Optional[int]
meta_match: Optional[dict[str, Any]]
```

Esto significa que esos campos son opcionales.

Una regla no tiene por qué usar todos los criterios a la vez.

Importación de BaseModel y Field

```
from pydantic import BaseModel, Field
```

Esta línea importa dos elementos de Pydantic.

BaseModel

`BaseModel` permite crear schemas de validación.

Las clases que heredan de `BaseModel` pueden validar datos automáticamente.

En este archivo se usa en:

```
class RuleCreate(BaseModel):
```

y:

```
class RuleOut(BaseModel):
```

Field

`Field` permite añadir restricciones a los campos.

Ejemplos:

```
name: str = Field(min_length=1, max_length=120)
severity_min: Optional[int] = Field(default=None, ge=0, le=10)
```

Esto permite controlar longitud, rangos numéricos y valores por defecto.

Definición de RuleCreate

```
class RuleCreate(BaseModel):
```

Esta línea define el schema de entrada para crear una regla.

Desglose:

```
class
```

Palabra clave para definir una clase.

```
RuleCreate
```

Nombre de la clase.

Indica que se usa para crear reglas.

```
(BaseModel)
```

Indica que hereda de Pydantic `BaseModel`.

Esto permite que FastAPI valide automáticamente el JSON enviado al endpoint `POST /rules`.

Campo `name`

```
name: str = Field(min_length=1, max_length=120)
```

Define el nombre de la regla.

Desglose:

```
name
```

Nombre del campo.

```
: str
```

Debe ser una cadena de texto.

```
Field(min_length=1, max_length=120)
```

Define restricciones de longitud.

Restricción `min_length=1`

```
min_length=1
```

Impide crear reglas con nombre vacío.

Ejemplo inválido:

```
{  
  "name": ""  
}
```

Restricción `max_length=120`

```
max_length=120
```

Impide que el nombre supere los 120 caracteres.

Esto está alineado con el modelo `Rule`, donde el campo `name` se define como:

```
String(120)
```

Campo `enabled`

```
enabled: bool = True
```

Define si la regla está activa o no.

Desglose:

```
enabled
```

Nombre del campo.

```
: bool
```

Debe ser booleano.

```
true
```

Valor por defecto.

Esto significa que, si el usuario no indica nada, la regla se crea activada.

Ejemplo:

```
{
  "name": "High severity auth events",
  "enabled": true
}
```

Si `enabled` es `false`, la regla se almacena pero no será evaluada en `POST /ingest`.

Campo `source`

```
source: Optional[str] = Field(default=None, max_length=64)
```

Define un filtro opcional por origen del evento.

Desglose:

```
source
```

Nombre del campo.

```
Optional[str]
```

Puede ser una cadena o `None`.

```
Field(default=None, max_length=64)
```

Valor por defecto `None` y longitud máxima 64.

Ejemplo:

```
{
  "source": "auth"
}
```

Una regla con `source = "auth"` solo coincidirá con eventos cuyo origen sea `auth`.

Si `source` es `None`, la regla no filtra por origen.

Campo `severity_min`

```
severity_min: Optional[int] = Field(default=None, ge=0, le=10)
```

Define una severidad mínima opcional.

Desglose:

```
severity_min
```

Nombre del campo.

```
Optional[int]
```

Puede ser entero o `None`.

```
Field(default=None, ge=0, le=10)
```

Valor por defecto `None` y rango permitido de 0 a 10.

Ejemplo:

```
{
  "severity_min": 5
}
```

Esta regla solo coincidirá con eventos cuya severidad sea igual o superior a 5.

Si `severity_min` es `None`, no se filtra por severidad.

Campo `contains`

```
contains: Optional[str] = Field(default=None, max_length=200)
```

Define un texto opcional que debe aparecer en el mensaje del evento.

Desglose:

```
contains
```

Nombre del campo.

```
Optional[str]
```

Puede ser una cadena o `None`.

```
Field(default=None, max_length=200)
```

Valor por defecto `None` y longitud máxima 200.

Ejemplo:

```
{
  "contains": "failed login"
}
```

La regla coincidirá con eventos cuyo mensaje contenga ese texto.

En `ingest.py`, la comparación se hace en minúsculas para evitar problemas con mayúsculas y minúsculas.

Comentario sobre `throttle`

```
# throttle por regla (segundos)
# None => usar DEFAULT_THROTTLE_SECONDS en el motor
# 0     => sin throttle (alertar siempre)
```

Estos comentarios explican la finalidad del campo `throttle_seconds`.

El `throttle` sirve para limitar la frecuencia de alertas generadas por una regla.

El comentario distingue tres situaciones:

```
None → usar un valor por defecto del motor
0    → sin throttle
>0   → aplicar throttle durante esos segundos
```

Punto importante: en el código actual de `ingest.py`, el `throttle` se aplica cuando:


```
rule.throttle_seconds is not None and rule.throttle_seconds > 0
```

Por tanto, en la implementación actual:

```
None → no entra en el bloque de throttle
0    → no entra en el bloque de throttle
>0   → aplica throttle
```

El comentario menciona un posible `DEFAULT_THROTTLE_SECONDS`, pero en el código de `ingest.py` que se ha revisado no aparece aplicado explícitamente. Es un buen punto a tener presente para no explicarlo mal.

Campo `throttle_seconds`

```
throttle_seconds: Optional[int] = Field(default=None, ge=0, le=86400)
```

Define el tiempo de throttle en segundos.

Desglose:

```
throttle_seconds
```

Nombre del campo.

```
Optional[int]
```

Puede ser entero o `None`.

```
Field(default=None, ge=0, le=86400)
```

Permite valores entre 0 y 86400.

86400 segundos equivalen a 24 horas.

Ejemplos:

```
0      → sin throttle
300    → 5 minutos
3600   → 1 hora
86400  → 24 horas
```

Si se configura a 300, la misma regla y grupo no deberían generar otra alerta hasta que pasen 300 segundos, siempre que exista `group_key`.

Comentario sobre `threshold`

```
# threshold: dispara alerta cuando hay N eventos que matchean en X segundos
```

Este comentario explica la finalidad de los campos de `threshold`.

El `threshold` permite generar una alerta no por un evento individual, sino cuando se acumulan varios eventos que cumplen una condición dentro de una ventana temporal.

Ejemplo:

```
5 eventos en 60 segundos
```

Esto es útil para detectar patrones repetitivos como múltiples fallos de login.

Campo `threshold_count`

```
threshold_count: Optional[int] = Field(default=None, ge=1, le=100000)
```

Define cuántos eventos deben coincidir para disparar la regla.

Desglose:

```
threshold_count
```

Nombre del campo.

```
Optional[int]
```

Puede ser entero o None .

```
Field(default=None, ge=1, le=100000)
```

Si se indica, debe estar entre 1 y 100000.

Ejemplo:

```
{
  "threshold_count": 5
}
```

Esto significa que se necesitan 5 eventos coincidentes.

Campo `threshold_seconds`

```
threshold_seconds: Optional[int] = Field(default=None, ge=1, le=86400)
```

Define la ventana temporal del threshold.

Desglose:

```
threshold_seconds
```

Nombre del campo.

```
Optional[int]
```

Puede ser entero o None .

```
Field(default=None, ge=1, le=86400)
```

Si se indica, debe estar entre 1 segundo y 86400 segundos.

Ejemplo:

```
{
  "threshold_seconds": 60
}
```

Esto significa que el sistema evaluará una ventana de 60 segundos.

Combinado con `threshold_count`:

```
{
  "threshold_count": 5,
  "threshold_seconds": 60
}
```

Significa:

```
5 eventos en 60 segundos
```

Comentario sobre `meta_match`

```
# match por meta (exacto). Ej: {"host":"kali","facility":"auth"}
```

Este comentario explica que `meta_match` permite definir coincidencias exactas sobre el campo `meta` del evento.

Ejemplo:

```
{
  "host": "kali",
  "facility": "auth"
}
```

Esto significa que la regla buscará eventos cuyo `meta` contenga esos pares clave-valor exactos.

Campo `meta_match`

```
meta_match: Optional[dict[str, Any]] = None
```

Define un filtro opcional sobre metadatos.

Desglose:

```
meta_match
```

Nombre del campo.

```
Optional[dict[str, Any]]
```

Puede ser un diccionario o `None`.

```
-
```

Valor por defecto.

Ejemplo:

```
{
  "meta_match": {
    "host": "server-01",
    "user": "admin"
  }
}
```

Durante la ingesta, el sistema compara cada clave y valor de `meta_match` con el campo `meta` del evento.

Si alguna clave no coincide, la regla no se aplica.

Definición de `RuleOut`

```
class RuleOut(BaseModel):
```

Define el schema de salida de una regla.

Este schema se utiliza como respuesta en:

```
POST /rules
GET /rules
```

A diferencia de `RuleCreate` , `RuleOut` incluye campos generados por la base de datos:

```
id
created_at
```

Campo `id`

```
id: int
```

Identificador único de la regla.

Lo genera PostgreSQL al insertar la regla.

No se envía al crear la regla, pero sí aparece en la respuesta.

Campo `name`

```
name: str
```

Nombre de la regla devuelto en la respuesta.

Campo `enabled`

```
enabled: bool
```

Indica si la regla está activa.

Este campo es importante porque solo las reglas activas se evalúan durante la ingesta.

Campo `source`

```
source: Optional[str]
```

Devuelve el filtro por origen de la regla.

Puede ser `None` .

Campo `severity_min`

```
severity_min: Optional[int]
```

Devuelve la severidad mínima configurada.

Puede ser `None` .

Campo `contains`

```
contains: Optional[str]
```

Devuelve el texto que debe contener el mensaje del evento.

Puede ser `None` .

Campo `throttle_seconds`

```
throttle_seconds: Optional[int]
```

Devuelve el throttle configurado en segundos.

Puede ser `None` .

Campo `threshold_count`

```
threshold_count: Optional[int]
```

Devuelve el número de eventos requeridos para el threshold.

Puede ser `None` .

Campo `threshold_seconds`

```
threshold_seconds: Optional[int]
```

Devuelve la ventana temporal del threshold.

Puede ser `None` .

Campo `meta_match`

```
meta_match: Optional[dict[str, Any]]
```

Devuelve las condiciones sobre metadatos configuradas en la regla.

Puede ser `None` .

Campo `created_at`

```
created_at: datetime
```

Devuelve la fecha de creación de la regla.

Este valor procede de la base de datos.

En el modelo `Rule` , está definido con:

```
server_default=func.now()
```

Configuración `model_config`

```
model_config = {"from_attributes": True}
```

Esta configuración permite que Pydantic construya `RuleOut` a partir de objetos SQLAlchemy.

Por ejemplo, en `rules.py` se devuelve:

```
return rule
```

`rule` es un objeto ORM de SQLAlchemy.

Gracias a:

```
model_config = {"from_attributes": True}
```

Pydantic puede leer:

```
rule.id
rule.name
rule.enabled
rule.source
rule.severity_min
rule.contains
rule.throttle_seconds
rule.threshold_count
rule.threshold_seconds
rule.meta_match
rule.created_at
```

y convertirlo en JSON.

Resultado final del archivo

Después de cargar este archivo, quedan disponibles dos schemas:

```
RuleCreate
RuleOut
```

Resumen:

```
RuleCreate
├─ name
├─ enabled
├─ source
├─ severity_min
├─ contains
├─ throttle_seconds
├─ threshold_count
├─ threshold_seconds
└─ meta_match

RuleOut
├─ id
├─ name
├─ enabled
├─ source
├─ severity_min
├─ contains
├─ throttle_seconds
├─ threshold_count
├─ threshold_seconds
├─ meta_match
└─ created_at
```

RuleCreate se usa para entrada.

RuleOut se usa para salida.

7 Relación con el flujo técnico del laboratorio

Este archivo participa en la fase de configuración de reglas.

Flujo de creación:

```
Cliente envía JSON
↓
POST /rules
↓
FastAPI valida con RuleCreate
↓
se crea modelo Rule
↓
se guarda en PostgreSQL
↓
se devuelve RuleOut
```

Después, esas reglas serán utilizadas por `ingest.py`:

```
POST /ingest
↓
se crea Event
↓
se consultan reglas activas
↓
Rule se evalúa contra Event
↓
si coincide, se crea Alert
```

Por tanto, este schema no evalúa reglas directamente, pero define qué condiciones puede configurar el usuario.

8 Errores típicos o puntos importantes

`name` es obligatorio

No se puede crear una regla sin nombre.

Además, debe tener entre 1 y 120 caracteres.

`enabled` por defecto es `True`

Si el usuario no envía `enabled`, la regla se crea activa.

Esto significa que empezará a evaluarse en `/ingest`.

`severity_min` debe estar entre 0 y 10

Valores como `-1` o `11` producirán error de validación.

`throttle_seconds` permite 0

El valor `0` es válido.

Según el comentario del código, significa sin throttle.

En la lógica actual de `ingest.py`, `0` no activa el bloque de throttle porque se exige:

```
rule.throttle_seconds > 0
```

`threshold_count` y `threshold_seconds` funcionan como pareja

Aunque el schema permite enviar solo uno de los dos, la lógica de `ingest.py` solo aplica threshold si ambos existen:

```
if rule.threshold_count is not None and rule.threshold_seconds is not None:
```

Por tanto, para definir un threshold funcional deben configurarse ambos.

`meta_match` permite reglas más flexibles

`meta_match` permite crear reglas sobre campos que no están en columnas fijas.

Ejemplo:

```
{
  "meta_match": {
    "user": "admin"
  }
}
```

Esto se compara con el campo `meta` del evento.

`from_attributes` es necesario para devolver modelos ORM

Como los endpoints devuelven objetos `Rule` de SQLAlchemy, `RuleOut` necesita:

```
model_config = {"from_attributes": True}
```

para convertirlos correctamente en JSON.

9 Comandos útiles relacionados

Crear regla simple por severidad:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "High severity events",
  "enabled": true,
  "severity_min": 5
}'
```

Crear regla por origen y texto:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "Auth failed login",
  "enabled": true,
  "source": "auth",
  "contains": "failed login"
}'
```

Crear regla con `meta_match`:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "Admin login failed",
  "enabled": true,
  "source": "auth",
  "meta_match": {
    "user": "admin"
  }
}'
```

Crear regla con `throttle`:


```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "Throttle auth failures",
  "enabled": true,
  "source": "auth",
  "contains": "failed login",
  "throttle_seconds": 300
}'
```

Crear regla con threshold:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "Five auth failures in one minute",
  "enabled": true,
  "source": "auth",
  "contains": "failed login",
  "threshold_count": 5,
  "threshold_seconds": 60
}'
```

Listar reglas:

```
curl http://localhost:8000/rules
```

Comprobar Swagger:

```
http://localhost:8000/docs
```

Probar importación del schema:

```
docker exec -it siem-api python -c "from app.schemas.rule import RuleCreate, RuleOut; print(RuleCreate, RuleOut)"
```